Comparative Research on GPU, its Operating Systems, and Application in AWS Cloud Platforms

Student Name:

Student ID: U2830528

Module Code: CN7029

Module Title: Computer Architecture and Networks

Table of Contents

1. Introduction 3

2. Task 1: Linux GPU Operating Systems Command-Line Tools 4

2.1 GPU Status and Monitoring (`dmesg | grep -i drm`) 5

2.2 Detect GPU Hardware Using PCI Scan (`lspci | grep -i vga`) 5

2.3 Check GPU Memory Usage (`grep -i mem /proc/meminfo`) 6

2.4 Monitor GPU/CPU Load (watch -n 2 ...) 7

2.5 GPU/Graphics Interrupts (`grep -i drm /proc/interrupts`) 8

2.6 CPU/GPU Temperature Monitoring (`sensors`) 8

2.7 Task 1 Summary 8

3. Task 2: AWS EC2 Implementation 9

3.1 EC2 Launch Page 9

3.2 EC2 Instance Configuration 10

3.3 Instance Type Selection 11

3.4 Key Pair Creation 12

3.5 Network Settings 12

3.6 Storage Configuration 13

3.7 Launch Status 14

3.8 Monitoring Status 14

3.9 Instance Stopping and Deletion 15

4. Task 3: Reflective Evaluation 17

4.1 GPU Diagnostics and Cloud Performance 17

4.2 Limitations of Virtual Environments 17

4.3 AWS Integration 18

4.4 Resource Management and Cost Efficiency 18

4.5 Evaluation Summary 18
5. Task 4: Conclusion 19
Summary of Work 19
Key Findings 19
Implications 19
Recommendations 20
Reference 21

List of Figures:

Figure 1 GPU status and monitoring 5
Figure 2 Check kernel messages for GPU/graphics info 6
Figure 3 Detect GPU hardware using PCI scan 6
Figure 4 Check GPU Memory Usage 7
Figure 5 Check GPU/CPU load using top or watch 8
Figure 6 Ec2 launch page 10
Figure 7 Ec2 instance config 11
Figure 8 Instance type 11
Figure 9 Key pair creation 12
Figure 10 Network setting 13
Figure 11 Storage config 13
Figure 12 Launch status 14
Figure 13 Monitoring status 15
Figure 14 Instance stopping 16
Figure 15 Instance deletion 16

1. Introduction

Modern computing environments are currently dependent on more and more high-performance hardware accelerators, specifically Graphics Processing Units (GPUs) in various systems, and used in many ways, including graphics rendering workloads and data-intensive computation workloads. This report is a comparative research study about GPU technology, integration with Linux-based operating systems (OS), and its use in the setting of cloud computing technology through Amazon Web Services (AWS). The main idea of this study is to explore the possibility of using Linux GPU command-line tools to monitor, diagnose, or perform analysis of the performance and how the findings can be applied to the cloud computing platforms.

The study will be organized into four tasks. Task 1 deals with analyzing the GPU monitoring and diagnostic commands in the Linux OS, indicating the capabilities and weaknesses of the GPU monitoring in the physical and virtualized systems. In Task 2, the specifics of implementing the cloud computing potential through AWS EC2 instances and the focus on the deployment, configuration, and monitoring of virtual servers will be presented. Task 3 presents a reflective assessment of the connection between Linux GPU command-line tools and cloud computing performance, whereas Task 4 wraps up the results of the findings, summarizing the main findings and implications.

This study will have three objectives: first, to gain a technical insight into the functioning of GPU hardware and Linux-based diagnostic software; second, to draw the comparison between the cloud deployment strategies and operational issues in the context of AWS and other systems; and third, to take the comparison between the combination of the methods of GPU monitoring and cloud infrastructure management. By using this organized presentation, the report shows the theoretical and practical expertise on the topic of GPU systems and cloud deployment.

2. Task 1: Linux GPU Operating Systems Command-Line Tools

The first phase of this study entailed a study of the use of Linux command-line Linux graphics card operating system tools in the detection of hardware, monitoring of hardware, and performance diagnostics of hardware. The task was performed on a Kali Linux virtual

environment, which mimics the case of a typical user in a virtualized environment, as well as raises the issue of hardware issues with virtual GPUs (Currieri et al., 2025).

2.1 GPU Status and Monitoring (`dmesg | grep -i drm`)

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
┌──(kali㉿kali)-[~]
└─$ # StudentID: U2830528 - Check kernel messages for GPU/graphics info
dmesg | grep -i drm

[    2.431457] ACPI: bus type drm_connector registered
[    2.848678] vmwgfx 0000:00:02.0: [drm] FIFO at 0x00000000f0000000 size is 2048 KiB
[    2.848693] vmwgfx 0000:00:02.0: [drm] VRAM at 0x00000000e0000000 size is 16384 KiB
[    2.848703] vmwgfx 0000:00:02.0: [drm] Running on SVGA version 2.
[    2.848710] vmwgfx 0000:00:02.0: [drm] Capabilities: rect copy, cursor, cursor bypass, cursor bypass 2, alpha cursor, extended fifo, pitchlock, irq mask
, gmr, traces, gmr2, screen object 2, command buffers, command buffers 2, gbobject,
[    2.848713] vmwgfx 0000:00:02.0: [drm] *ERROR* vmwgfx seems to be running on an unsupported hypervisor.
[    2.848720] vmwgfx 0000:00:02.0: [drm] *ERROR* This configuration is likely broken.
[    2.848724] vmwgfx 0000:00:02.0: [drm] *ERROR* Please switch to a supported graphics device to avoid problems.
[    2.848728] vmwgfx 0000:00:02.0: [drm] DMA map mode: Caching DMA mappings.
[    2.848836] vmwgfx 0000:00:02.0: [drm] Legacy memory limits: VRAM = 16384 KiB, FIFO = 2048 KiB, surface = 507904 KiB
[    2.848840] vmwgfx 0000:00:02.0: [drm] MOB limits: max mob size = 131072 KiB, max mob pages = 4096
[    2.848842] vmwgfx 0000:00:02.0: [drm] Max GMR ids is 8192
[    2.848844] vmwgfx 0000:00:02.0: [drm] Max number of GMR pages is 1048576
[    2.848845] vmwgfx 0000:00:02.0: [drm] Maximum display memory size is 16384 KiB
[    2.852076] vmwgfx 0000:00:02.0: [drm] Screen Target display unit initialized
[    2.852379] vmwgfx 0000:00:02.0: [drm] Fifo max 0x00200000 min 0x00001000 cap 0x00000355
[    2.853055] vmwgfx 0000:00:02.0: [drm] Using command buffers with DMA pool.
[    2.853075] vmwgfx 0000:00:02.0: [drm] Available shader model: Legacy.
[    2.854538] [drm] Initialized vmwgfx 2.21.0 for 0000:00:02.0 on minor 0
[    2.857699] fbcon: vmwgfxdrmfb (fb0) is primary device
[    2.862818] vmwgfx 0000:00:02.0: [drm] fb0: vmwgfxdrmfb frame buffer device
[    5.305706] systemd[1]: modprobe@drm.service - Load Kernel Module drm was skipped because of an unmet condition check (ConditionKernelModuleLoaded=!drm)
.
[    8.177453] 10:13:08.129838 main     Executable: /usr/bin/VBoxDRMClient
[    8.186089] 10:13:08.139141 main     VBoxDRMClient: found compatible device: /dev/dri/renderD128
[    8.191180] 10:13:08.144040 main     VBoxDRMClient: starting deferred resize events delivery
[    8.194203] 10:13:08.147035 main     VBoxDRMClient: IPC server socket access granted to all users
[    8.443425] 10:13:08.396174 Timer    VBoxDRMClient: push screen layout data of 1 display(s) to DRM stack, fPartialLayout=false, rc=VERR_INV
[    8.444726] 10:13:08.397460 Timer    VBoxDRMClient: Error: Failed to push display change as requested by host, rc=VERR_INVALID_PARAMETER
[  301.978838] 10:18:02.078579 IpcCLT-1012 VBoxDRMClient: IPC client connection started
[  304.497497] 10:18:04.598027 IpcCLT-1012 VBoxDRMClient: thread IpcCLT-1012 notified IPC Client that display 0 is now primary, rc=VINF_SUCCES

┌──(kali㉿kali)-[~]
└─$
```

[redacted]

[redacted]

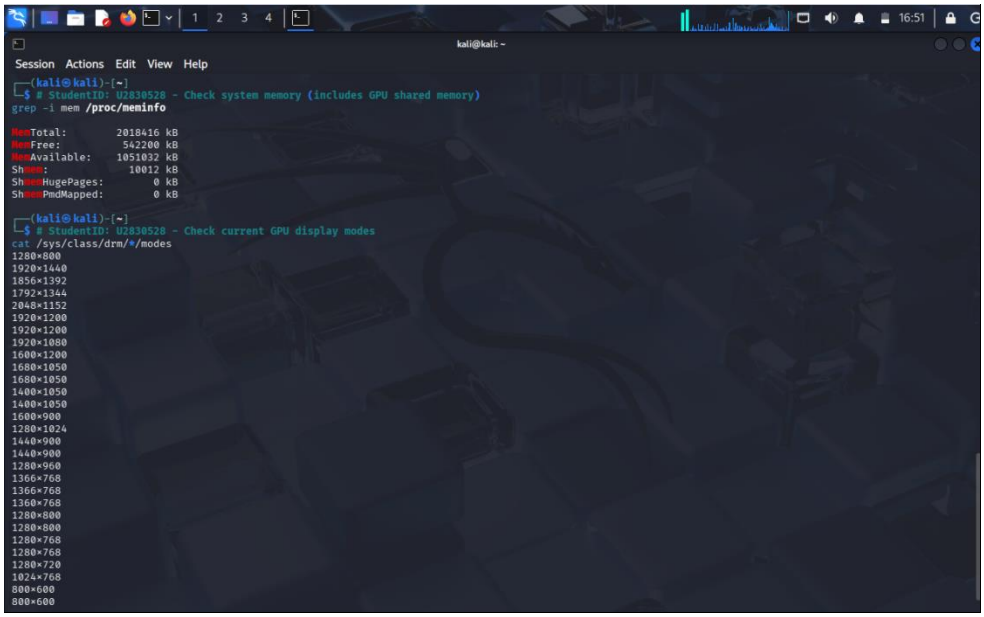
```
┌──(kali㉿kali)-[~]
└─$ # StudentID: U2830528 - Detect GPU hardware using PCI scan
lspci | grep -i vga

00:02.0 VGA compatible controller: VMware SVGA II Adapter

┌──(kali㉿kali)-[~]
└─$
```

[redacted]

[redacted]

[redacted]

[redacted]

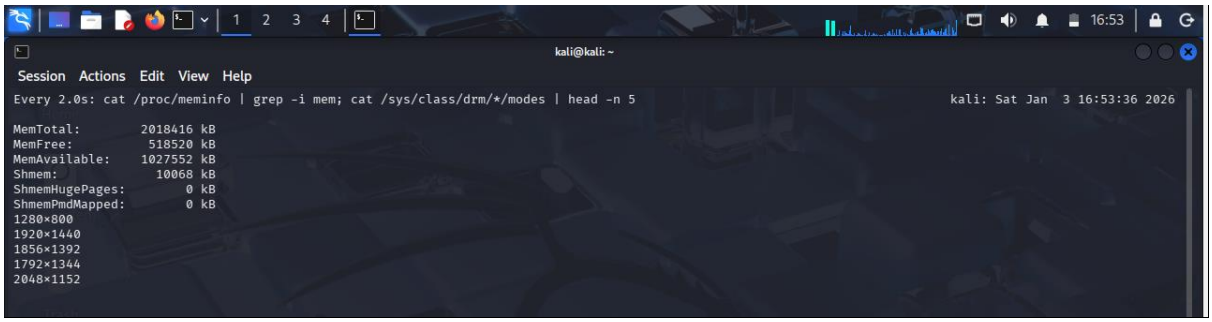


[redacted]

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

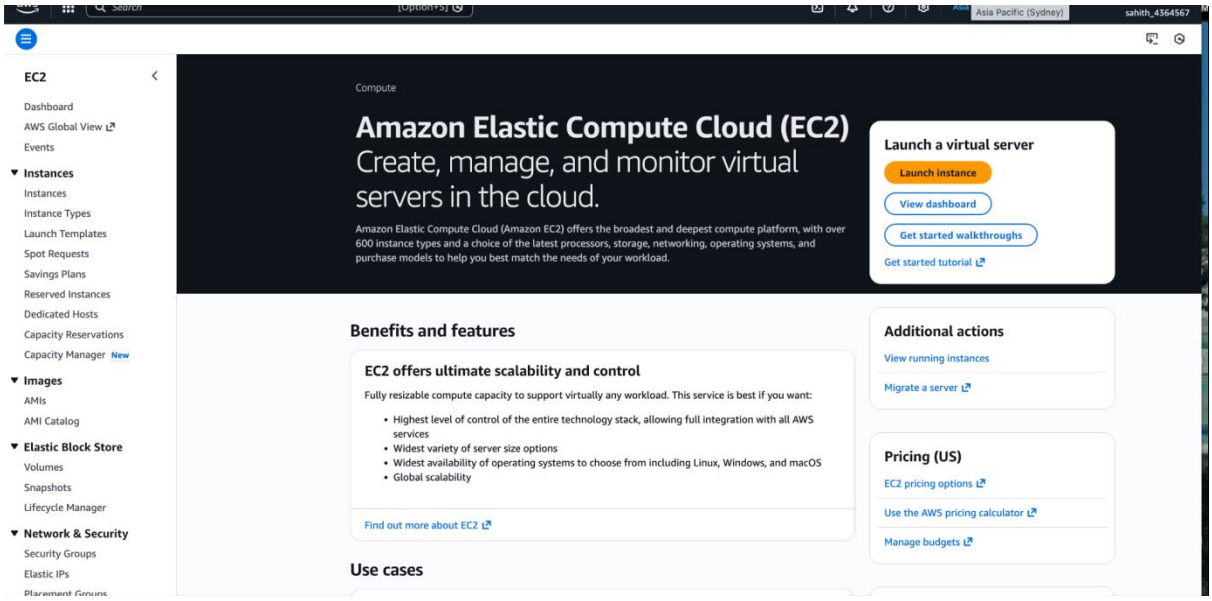
[redacted]

[redacted]

[redacted]

[redacted]

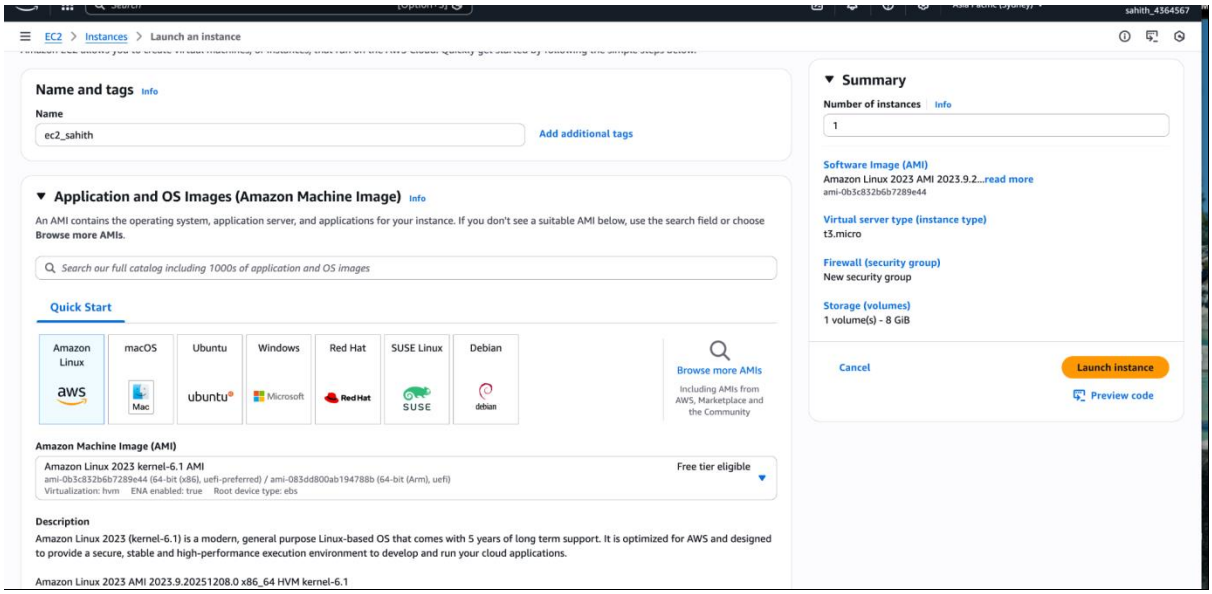
[redacted]



[redacted]

[redacted]

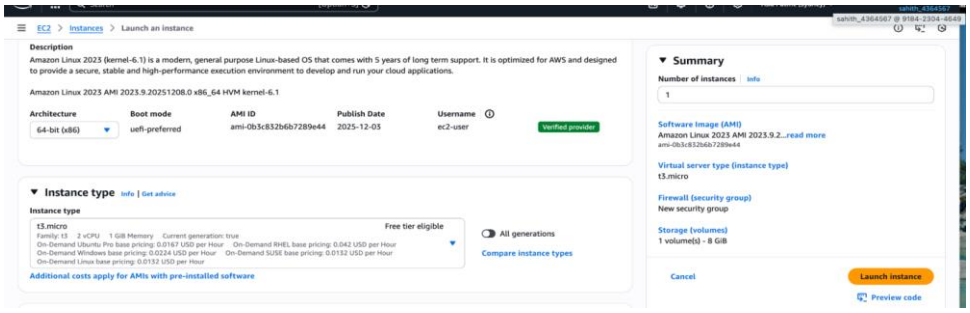
[redacted]



[redacted]

[redacted]

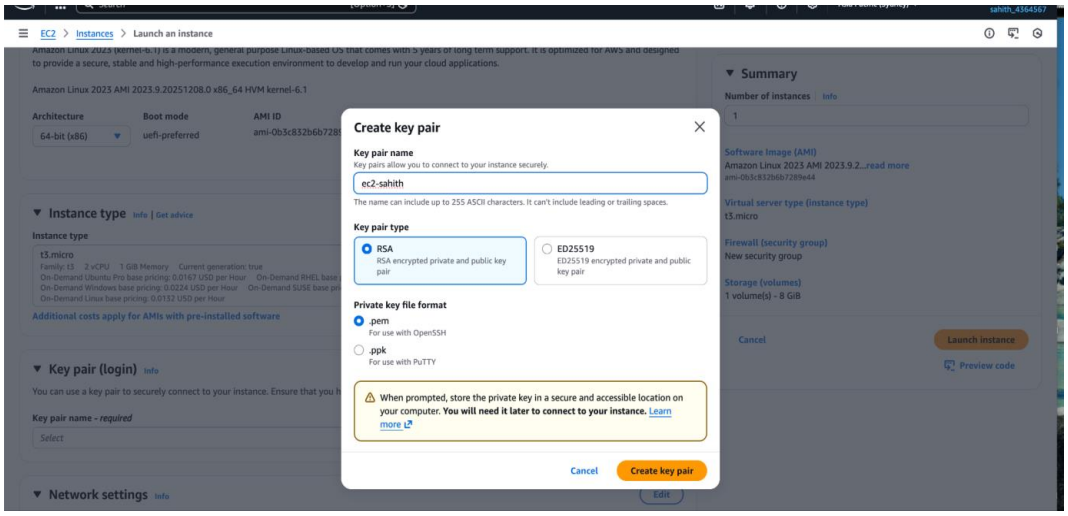
[redacted]



[redacted]

[redacted]

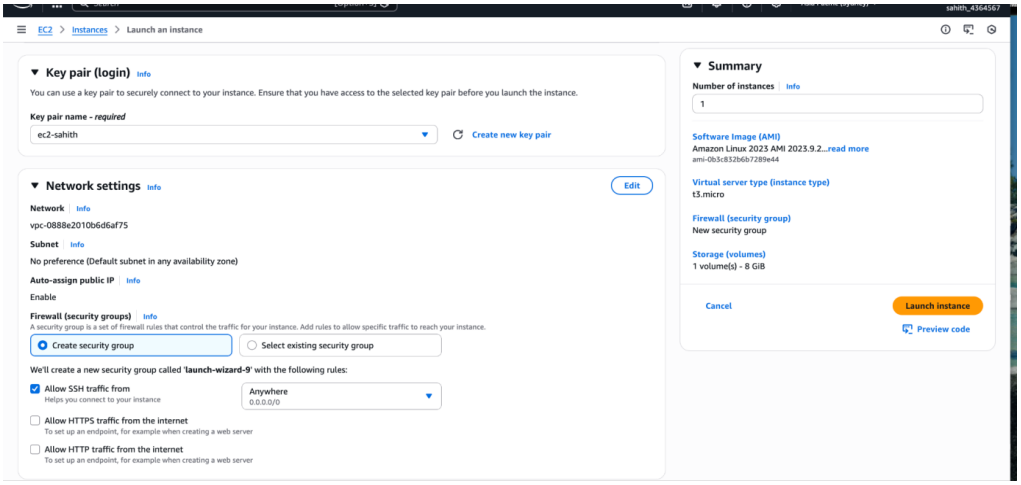
[redacted]



[redacted]

[redacted]

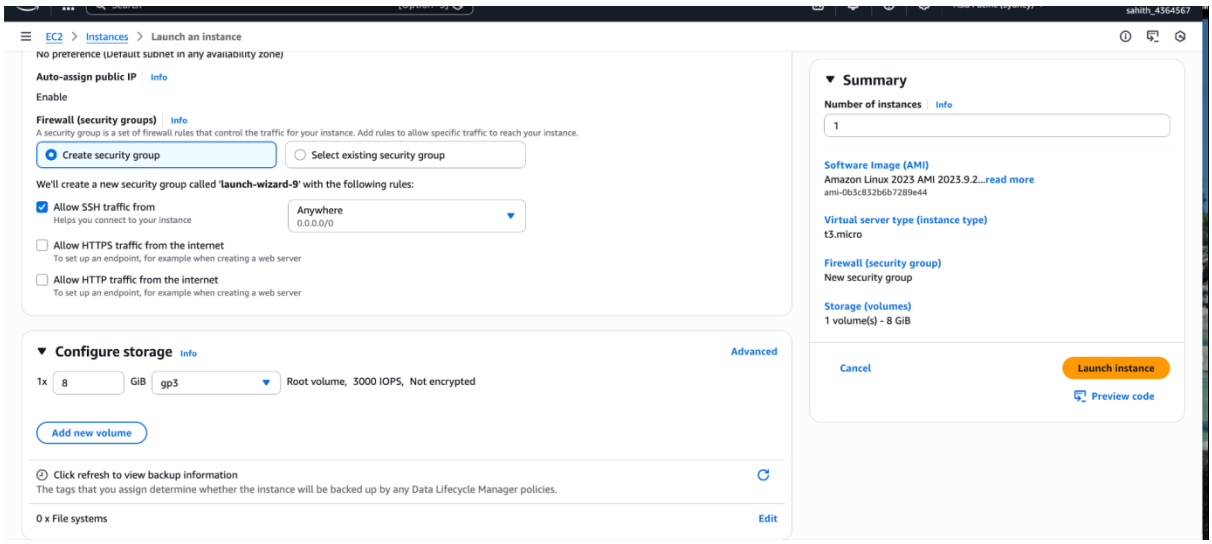
[redacted]



[redacted]

[redacted]

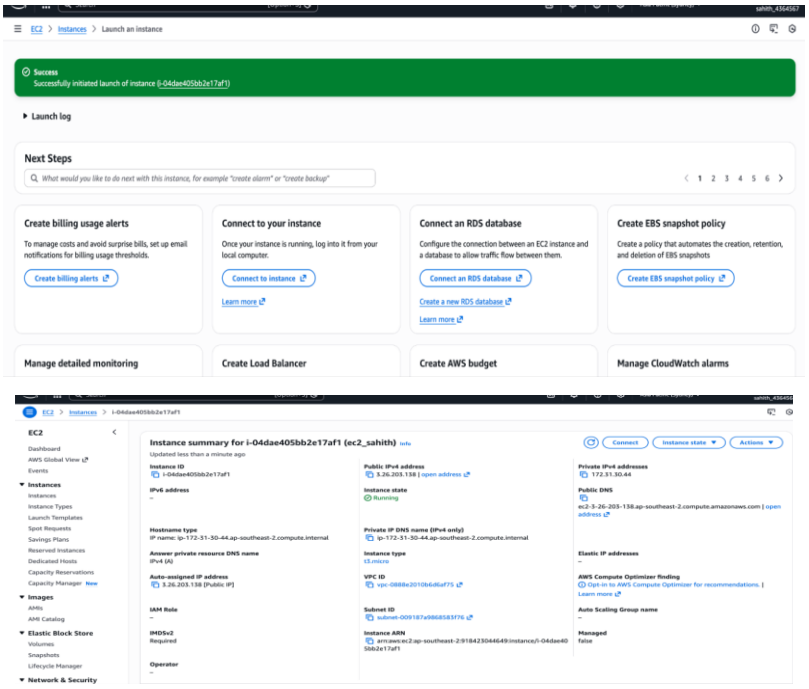
[redacted]



[redacted]

[redacted]

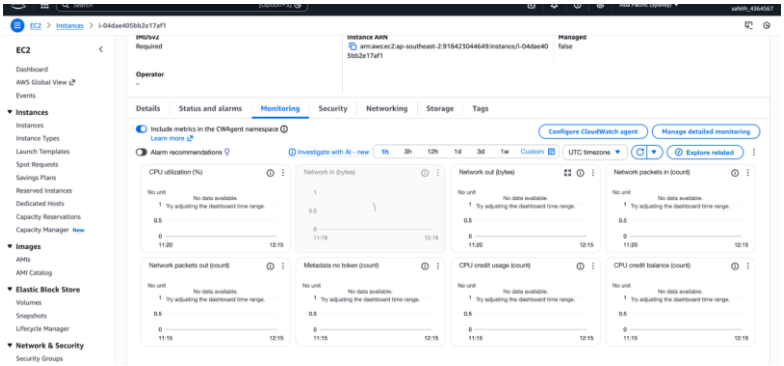
[redacted]

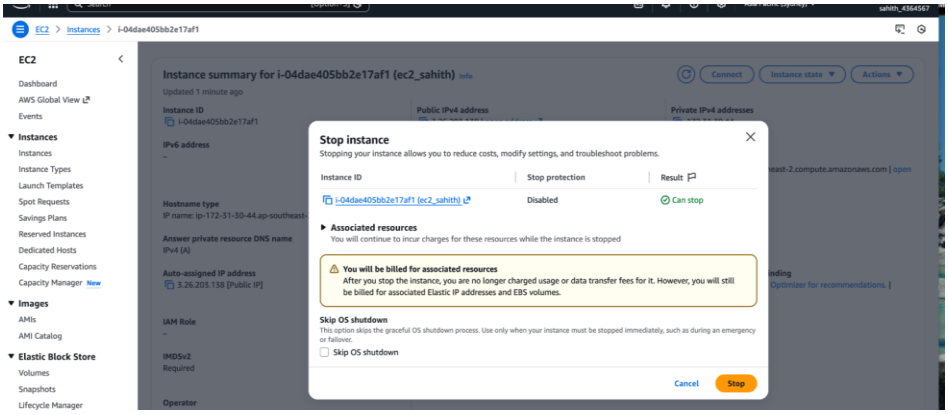


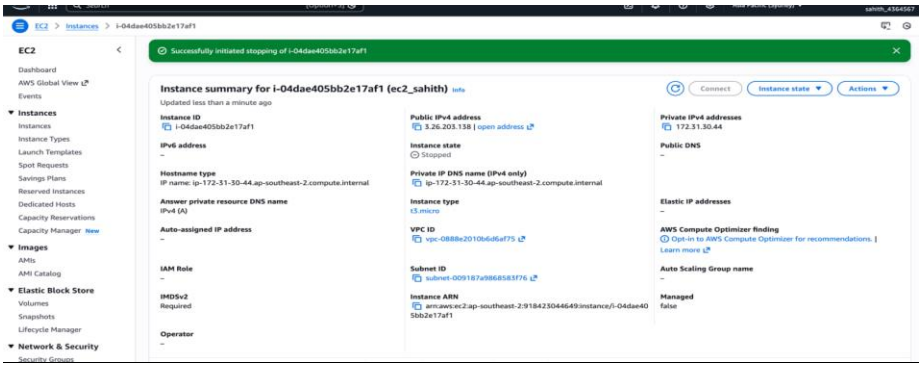
[redacted]

[redacted]

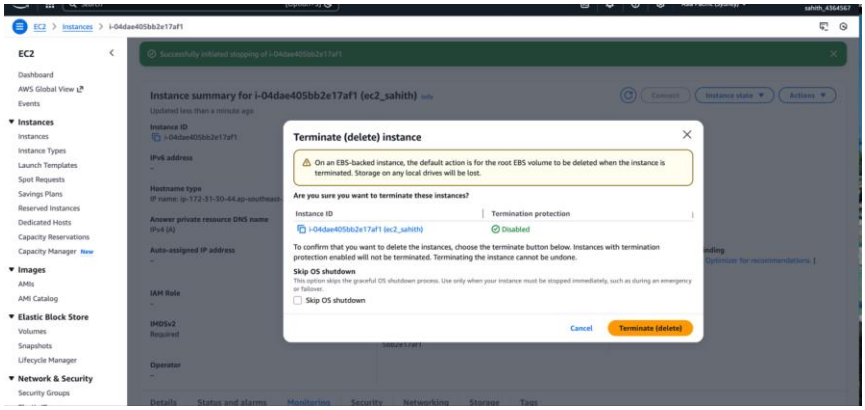
[redacted]

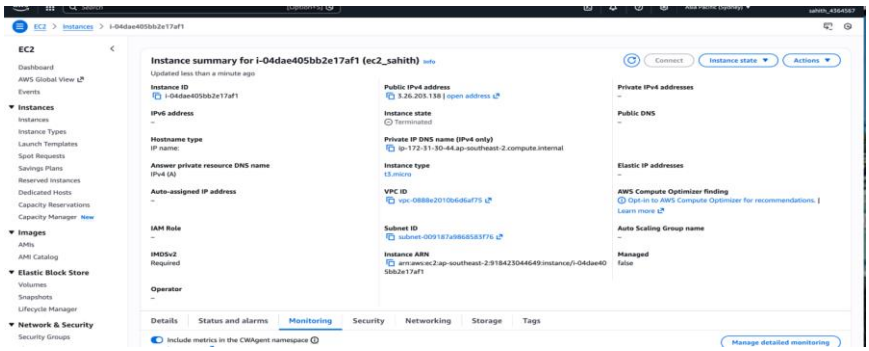

EC2 > Instances > i-04dae405bb2e17af1
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
IMDSv2
Required
Operator
--
Instance ARN
arn:aws:ec2:ap-southeast-2:918423044649:instance/i-04dae405bb2e17af1
Managed
false
Details
Status and alarms
Monitoring
Security
Networking
Storage
Tags
Include metrics in the CWAgent namespace
Learn more
Configure CloudWatch agent
Manage detailed monitoring
Alarm recommendations
Investigate with AI - new
1h
3h
12h
1d
3d
1w
Custom
UTC timezone
Explore related
CPU utilization (%)
Network in (bytes)
Network out (bytes)
Network packets in (count)
Network packets out (count)
Metadata no token (count)
CPU credit usage (count)
CPU credit balance (count)
No unit
No data available.
Try adjusting the dashboard time range.





[redacted]





[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]